

ITM SKILLS UNIVERSITY

B.Tech CSE (2025–29)

PROJECT DOCUMENTATION

Bank Account Interest Calculator

Console and Swing GUI application in Java

Subject: Java Programming

Project No.: 160

Program: B.Tech CSE (2025–29)

University: ITM Skills University

Semester: Semester III

Submitted By: Sameer Shaikh

Roll Number: 150096725026

Cohort: Jeff Bezos

1. Introduction

This project is a Bank Account Interest Calculator written in Java. The user enters the account holder's name, account number, balance, interest rate, duration in years and the type of account. The program adds a bonus rate depending on the account type, calculates simple interest and the maturity balance, and prints an account summary.

The project has two source files. `BankInterestCalculator.java` holds the calculation methods and a console version of the program that reads input with `Scanner`. `BankInterestGUI.java` is a Swing window (it extends `JFrame`) that does the same job with text fields and buttons and calls the calculation methods of the first file.

2. Problem Statement

Working out the interest on a deposit by hand is repetitive, and the result changes with the account type. A program is needed that takes the account details, checks them, applies the rules for the selected account type, and shows the interest earned and the final maturity amount.

3. Objectives

- Take account details (name, account number, balance) and validate them.
- Let the user choose Savings, Fixed Deposit or Recurring Deposit using a `switch` statement (console) or a combo box (GUI).
- Apply account-specific bonus rates using `if-else` statements.
- Calculate simple interest, $(P \times R \times T) / 100$, and maturity balance = principal + interest.
- Display an account summary and handle wrong input with loops and `try-catch`.
- Provide a Swing GUI that reuses the same calculation methods.

4. Features and Technologies

- Name must contain only letters and spaces; account number only digits (stored as a `String`, so leading zeros are kept).
- Balance must not be negative, rate must be between 0 and 100, duration must be greater than 0 (decimals such as 1.5 are allowed).
- Three account types with their own bonus rules.
- Formatted summary with two decimal places; a Clear button in the GUI resets all fields.

Item	Used in the project
Language	Java, standard libraries only (no external jars, no build tool)
Console	<code>java.util.Scanner</code> , <code>System.out.println</code> , <code>System.out.printf</code> , <code>InputMismatchException</code>
GUI	<code>javax.swing</code> (<code>JFrame</code> , <code>JPanel</code> , <code>JLabel</code> , <code>JTextField</code> , <code>JComboBox</code> , <code>JButton</code> , <code>JOptionPane</code>) and <code>java.awt</code> (<code>BorderLayout</code> , <code>GridLayout</code> , <code>Color</code> , <code>Font</code>)
Requirements	JDK to compile and run, any operating system. The GUI needs a desktop display; the console version only needs a terminal. No database or network is used.

5. System Architecture and Structure

The calculation logic is kept in static methods of `BankInterestCalculator`. The console program and the GUI are two separate front-ends that call these methods. The GUI does not call the console `main()` and does not use `Scanner`.

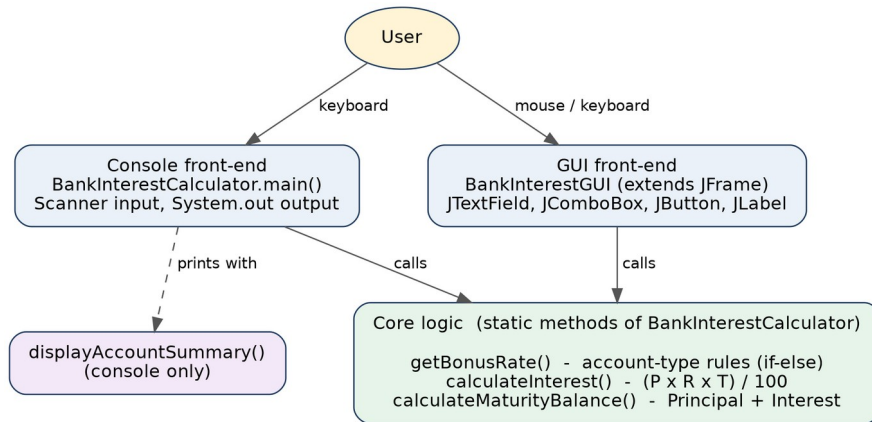


Figure 1: Architecture of the project

File	Contents
<code>BankInterestCalculator.java</code>	Calculation methods, bonus rules, summary printer and the console <code>main()</code>
<code>BankInterestGUI.java</code>	Swing user interface with its own <code>main()</code> ; depends on <code>BankInterestCalculator</code>

Compile both files together and run whichever version is needed:

```

javac BankInterestCalculator.java BankInterestGUI.java
java BankInterestCalculator      (console)
java BankInterestGUI            (GUI)
  
```

6. Workflow

Both versions follow the same steps. In the console version each input is read inside a loop and asked again until it is valid. In the GUI the same checks run when the Calculate Interest button is clicked; if a check fails, an error dialog appears and the summary is not changed.

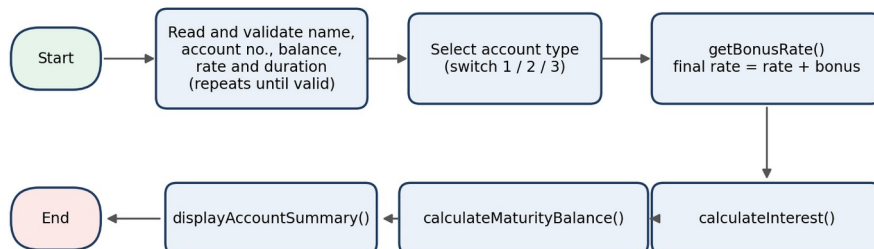


Figure 2: Program flow

Console version, step by step

1. The name and account number are read with `nextLine()` and checked with regular expressions.
2. Balance, rate and duration are read with `nextDouble()` inside `try-catch` blocks and range-checked.

3. A menu is shown and `nextInt()` reads the choice; a switch stores the account type name.
4. `getBonusRate()` gives the bonus, and the final rate is the entered rate plus the bonus.
5. `calculateInterest()` and `calculateMaturityBalance()` give the results, which `displayAccountSummary()` prints.

GUI version, step by step

1. `main()` creates the window through `SwingUtilities.invokeLater()`; the constructor builds the form and sets the summary to default values.
2. Clicking Calculate Interest runs `calculateAndShow()`, which reads the fields and checks them in order: name, account number, balance, rate, duration.
3. On the first wrong value `showError()` shows a dialog and returns focus to that field.
4. If everything is valid, the same three calculation methods are called and the ten summary labels are filled with two-decimal text.
5. Clicking Clear runs `clearAll()`, which empties the fields and resets the summary without closing the window.

7. Classes and Methods

`BankInterestCalculator` contains only static methods and no fields. `BankInterestGUI` extends `JFrame`; its private fields are the five text fields, the combo box, ten result labels and the colours.

Method	Class	Purpose
<code>calculateInterest(principal, rate, years)</code>	Calculator	Returns $(\text{principal} * \text{rate} * \text{years}) / 100$
<code>calculateMaturityBalance(principal, interest)</code>	Calculator	Returns $\text{principal} + \text{interest}$
<code>getBonusRate(accountType, balance, years)</code>	Calculator	Returns the bonus rate for the account type, or 0
<code>displayAccountSummary(...)</code>	Calculator	Prints the summary with <code>printf</code> (console only)
<code>main(String[] args)</code>	Calculator	Reads and validates input, calls the methods above, prints the result
<code>BankInterestGUI()</code> constructor	GUI	Builds the window: header, Account Details card, Account Summary card, button listeners
<code>readNumber(String)</code>	GUI	Converts text to double; throws <code>NumberFormatException</code> for invalid text, NaN or infinity
<code>calculateAndShow()</code>	GUI	Button handler: validates inputs, calls the three calculation methods, fills the summary labels
<code>showError(String, JTextField)</code>	GUI	Shows a <code>JOptionPane</code> error and puts focus on the wrong field
<code>clearAll(), resetSummary()</code>	GUI	Empty the fields and restore the default summary values
<code>styleField, makeLabel, addSummaryRow</code>	GUI	Small helpers for fonts, borders and summary rows
<code>main(String[] args)</code>	GUI	Starts the window using <code>SwingUtilities.invokeLater</code>

The GUI window uses `BorderLayout` for the frame, with a header at the top and two cards side by side in a `GridLayout(1, 2)`. The left card has the form (`GridLayout(6, 2)`) and the Calculate Interest and Clear buttons. The right card shows the Maturity Balance in large text and nine summary rows below it.

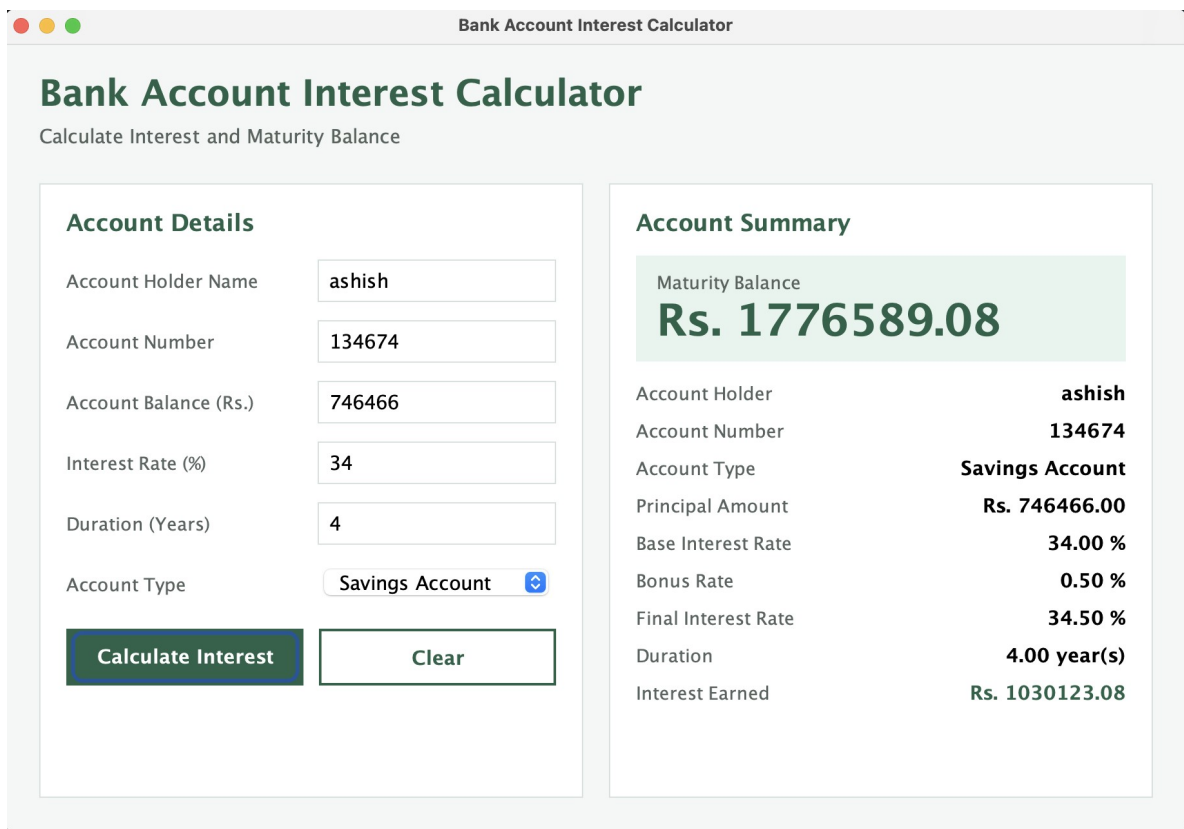


Figure 3: GUI Layout of the Bank Account Interest Calculator

8. Java Concepts Used

Concept	Where it is used
Static methods, parameters, return values	All calculation methods in BankInterestCalculator
Decision making	if-else chains in getBonusRate(); switch for account type in the console main()
Loops	while loops that repeat each input until it is valid
Strings and regex	matches("[A-Za-z]+") for the name, matches("[0-9]+") for the account number
Exception handling	InputMismatchException in the console, NumberFormatException in the GUI
Array and generics	String[] accountTypes fills a JComboBox<String>
Inheritance	BankInterestGUI extends JFrame
Encapsulation	Private fields and methods in BankInterestGUI
Interfaces, anonymous classes	Anonymous ActionListener for the two buttons and Runnable for invokeLater
Event handling and layouts	addActionListener, BorderLayout, GridLayout

The project does not use collections, file handling, a database, or threads created by the programmer. invokeLater only places the GUI creation on Swing's event dispatch thread.

9. Important Code Logic

9.1 Formulas

```
double interest = (principal * rate * years) / 100; // calculateInterest()
double maturity = principal + interest;           // calculateMaturityBalance()
double finalRate = rate + bonus;                  // bonus from getBonusRate()
```

Simple interest is used, so interest is not added back to the principal during the term. Example: Savings, balance 100000, rate 4 %, 2 years. The bonus is 0.5 % (balance is at least 50000), so the final rate is 4.5 %, interest = $(100000 \times 4.5 \times 2) / 100 = 9000$ and maturity = 109000.

9.2 Bonus rules

These rules are assumptions made for this project (the source code says so in a comment); they are not real bank rates.

Account type	Condition	Bonus
Savings Account	balance $\geq$ 50000 / otherwise	0.5 % / 0 %
Fixed Deposit Account	years $\geq$ 5 / years $\geq$ 2 / otherwise	1.0 % / 0.5 % / 0 %
Recurring Deposit Account	years $\geq$ 3 / otherwise	0.25 % / 0 %

9.3 Input validation

Input	Accepted when
Name	not empty, letters and spaces only
Account number	digits only
Balance	a number, not negative (0 is accepted)
Interest rate	a number from 0 to 100
Duration	a number greater than 0
Account type (console)	1, 2 or 3

In the console version a wrong data type makes `nextDouble()` throw `InputMismatchException`. The catch block prints an error and calls `sc.next()` to remove the bad token from the buffer; without this the loop would keep failing on the same input.

```
try {
    balance = sc.nextDouble();
    if (balance < 0) { System.out.println("Error: Balance cannot be negative."); }
    else { break; }
} catch (InputMismatchException e) {
    System.out.println("Error: Please enter a valid number for account balance.");
    sc.next(); // throw away the wrong input
}
```

10. Testing

The following test cases are based on the calculation and validation logic implemented in the Java source files. The expected values are calculated from the implemented formulas and bonus rules. Actual execution results should be verified when demonstrating the project.

ID	Account type	Balance	Rate %	Years	Bonus %	Interest	Maturity
T01	Savings	100000	4	2	0.50	9000.00	109000.00
T02	Savings	40000	4	3	0.00	4800.00	44800.00
T03	Savings (boundary)	50000	4	1	0.50	2250.00	52250.00
T04	Fixed Deposit	200000	6	5	1.00	70000.00	270000.00
T05	Fixed Deposit	100000	6	2	0.50	13000.00	113000.00
T06	Fixed Deposit	100000	6	1.5	0.00	9000.00	109000.00
T07	Recurring Deposit	60000	5	3	0.25	9450.00	69450.00
T08	Recurring Deposit	60000	5	2	0.00	6000.00	66000.00
T09	Savings (zero balance)	0	5	2	0.00	0.00	0.00
T10	Savings (decimals)	75000.50	6.5	2.5	0.50	13125.09	88125.59

Invalid input was also tested. In every case below the program printed the matching error message, asked again, and finished normally once a valid value was typed.

Input	Invalid values tried
Name	empty line, John123
Account number	12AB, 12 - 34
Balance	abc, -500
Interest rate	-1, 101, xyz
Duration	0, -2, abc
Account type	5, 0, abc

The GUI uses the same calculation methods, so its results for valid input are the same. For example, Savings, Rs. 100000, 4 %, 2 years should show bonus 0.50 %, final rate 4.50 %, interest Rs. 9000.00 and maturity Rs. 109000.00. An empty name, a negative balance, a rate above 100 or a duration of 0 should each open an "Input Error" dialog.

11. Output

Console run for test T01 (the text after each prompt is the user's input):

```
Enter Account Holder Name: Rahul Sharma
Enter Account Number: 1234567890
Enter Account Balance (Rs.): 100000
Enter Interest Rate (% per year): 4
Enter Duration (in years): 2
Select Account Type: 1. Savings 2. Fixed Deposit 3. Recurring Deposit
Enter your choice: 1
Bonus rate of 0.5% applied.
=====
ACCOUNT SUMMARY
=====
Account Holder      : Rahul Sharma
Account Number      : 1234567890
Account Type        : Savings Account
Principal Amount    : Rs. 100000.00
Base Interest Rate  : 4.00 %
Bonus Rate          : 0.50 %
Final Interest Rate : 4.50 %
Duration            : 2.00 year(s)
Interest Earned     : Rs. 9000.00
Maturity Balance    : Rs. 109000.00
=====
```

Invalid interest rates being rejected (test V04):

```
Enter Interest Rate (% per year): -1
Error: Interest rate cannot be negative.
Enter Interest Rate (% per year): 101
Error: Interest rate cannot be greater than 100%.
Enter Interest Rate (% per year): xyz
Error: Please enter a valid number for interest rate.
Enter Interest Rate (% per year): 4
```

[Paste GUI screenshot here: window after clicking Calculate Interest]

Figure 4: GUI window showing the account summary

12. Advantages and Limitations

Advantages

- The formulas and bonus rules are written once and used by both the console and the GUI.
- Every input is checked, and wrong data types do not crash the console program.
- Boundary values such as balance 50000 and 2 years were tested.
- Uses only standard Java, so it runs without extra setup.

Limitations

- Only simple interest is calculated; there is no compound interest.

- Recurring Deposit is calculated like the other accounts, as one amount. Monthly instalments are not handled.
- Bonus rules are fixed in the code, and account types are matched by their text, so a spelling difference would give no bonus.
- Validation code is repeated in the console and GUI classes.
- The name check rejects names with hyphens or apostrophes.
- Results are not saved anywhere, and only one account is handled at a time.
- No JUnit or other automated unit-test code is included in the project. The documented test cases are functional test scenarios for validating the implemented input validation and calculation logic.

13. Future Scope

- Add compound interest and monthly instalments for Recurring Deposit.
- Use an enum for account types and keep validation in one shared method.
- Save account records using files or a database, and support several accounts with an ArrayList.
- Use BigDecimal for money values and add JUnit tests.

14. Conclusion

The project calculates interest and maturity balance for three account types and checks the user's input at every step. Keeping the calculation methods separate from the user interface let the console and GUI versions share the same logic. It covers methods, if-else, switch, loops, regular expressions, exception handling, an array, inheritance from JFrame, event handling and Swing layouts. Interest is simple interest only and the bonus rules are assumptions, which are the main points to extend later.

15. References

1. Oracle, The Java Tutorials: Creating a GUI With Swing.
<https://docs.oracle.com/javase/tutorial/uiswing/>
2. Oracle, The Java Tutorials: Exceptions.
<https://docs.oracle.com/javase/tutorial/essential/exceptions/>
3. Oracle, Java SE API documentation (Scanner, JFrame, Double).
<https://docs.oracle.com/en/java/javase/21/docs/api/>